D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji.

(An Empowered Autonomous Institute, Affiliated to

Shivaji University, Kolhapur)

Department of Computer Science Engineering(AI)

2025-2026



Project Synopsis On

PostAura – AI Based Social Media Manager

The Guidance Of Mrs. Vanchala Sutar

Submitted By:

Sr.	PRN	Name
1.	22UAI067	Sai Anil Mahajan
2.	22UAI068	Shreyash Deepak Malani
3.	22UAI015	Prachi Popat Chougule
4.	22UAI062	Mehfooz Maqsood Kudachikar
5.	22UAI066	Abhinandan Guruprasad Madiwal

Prof.Shital Gaikwad

Prof.Dr.S.K.Shirgave

Project Guide **Head of Department**

INDEX

Sr. No.	Title	Page No.
1	Introduction	3
2	Overall Description	4
3	Specific Requirements	10
4	Software Design Document	20
5	References	28
6	Code and Outputs	32

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for the **"Post-Aura" AI-Based Social Media Manager** system. This document serves as a reference for developers, testers, and stakeholders to ensure a clear understanding of system objectives, scope, and behavior.

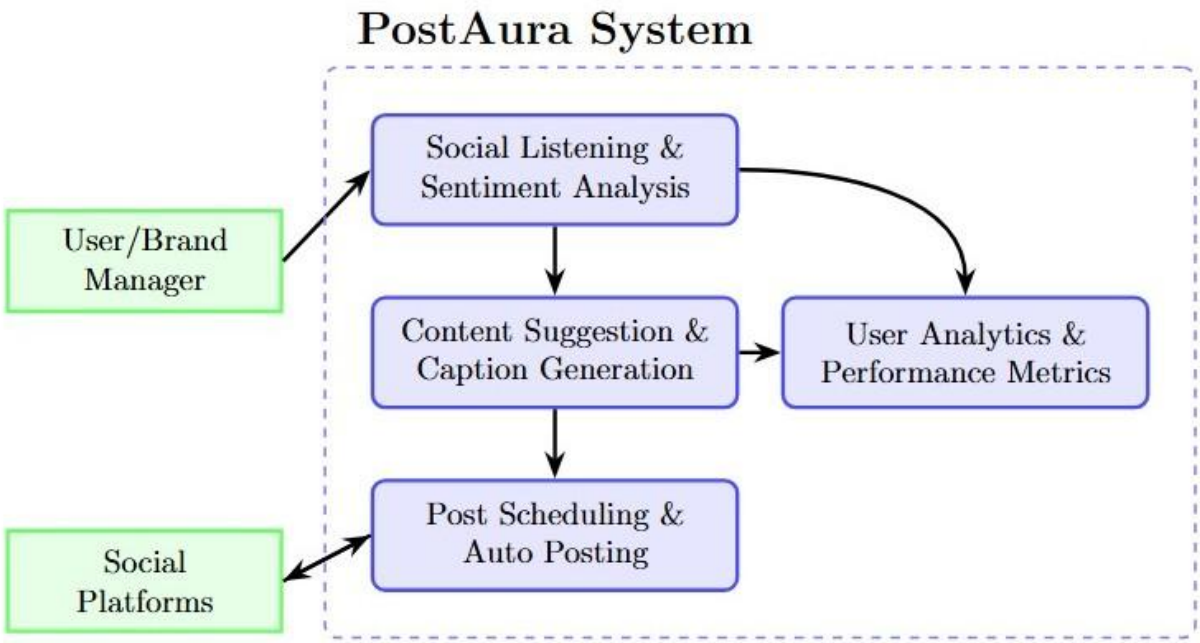
1.2 Scope

"Post-Aura" automates the process of managing social media accounts by collecting posts and comments from various platforms, analyzing sentiment, identifying trends, generating captions, scheduling posts, and displaying analytics.
The system will support multiple social media platforms (e.g., Twitter/X, Facebook, Instagram) through API integration and provide a user-friendly dashboard for visualization and control.

2.Overall description

2.1. Product perspective

2.1.1 Block Diagram



2.1.2 Hardware Requirements

- Processor: Intel Core i5/i7 or AMD equivalent (Quad-Core or higher)
- RAM: Minimum 8 GB (Recommended 16 GB for AI model integration and multitasking)
- Storage: 500 GB HDD or 256 GB SSD **(**or higher) for faster read/write operations
- GPU**:** NVIDIA GPU (optional but recommended for training or fine-tuning NLP/ML models)

- Internet Connection: Stable broadband or Wi-Fi connection for API access, data fetching, and cloud integration

2.1.2 Software Requirements

1. Frontend Development

- Technologies: HTML ,Tailwind CSS ,React.js.
- Purpose: To build a responsive and interactive dashboard interface.
- Package Manager**:** Node.js package manager (npm) for dependency management.

2. Backend Development

- Framework**:** Node.js with Express.js.
- Authentication**:** OAuth 2.0 for secure user login and social media API access.
- API Handling**:** RESTful APIs for communication between frontend and backend.

3. Database Management

MongoDB**:** For flexible and scalable storage of unstructured data (e.g., posts, comments, analytics logs)

4. AI & NLP Services

This process involves extracting real-time social media or web data using BeautifulSoup or Scrapy, followed by text preprocessing to clean and prepare the data.
The refined text is then analyzed using BERT-based sentiment models to detect emotions or opinions.
Finally, the system generates sentiment insights that help visualize public mood, trends, and engagement patterns.

5. Analytics & Visualization

Chart.js, D3.js, and Recharts are powerful charting libraries used for creating data visualizations and interactive analytics graphs that display insights such as engagement trends, sentiment distributions, and performance metrics. Additionally, Analytics API is integrated to track website traffic and social engagement metrics, providing detailed insights into user interactions and overall digital performance.

6. Web Scraping Tools

BeautifulSoup and Scrapy are powerful Python libraries used for efficient content extraction from websites and social media platforms. They enable automated collection of posts, comments, and other relevant textual data for analysis. Additionally, Requests (in Python) and Axios (in Node.js) are used as HTTP clients to perform REST API calls and fetch structured data directly from online APIs or web servers, ensuring smooth and real-time data retrieval for processing and analysis.

2.1.4 Cloud and Hosting Facilities

The project uses AWS S3 (Amazon Simple Storage Service) as the primary cloud platform for hosting and data storage. It provides a reliable, scalable, and secure environment to store and manage all project-related files, including images, videos, and documents. AWS S3 ensures high availability, fast access, and data backup, making it an ideal solution for handling both static assets and dynamic content efficiently.

2.1.5 APIs and Integrations

External API Integrations

The system integrates multiple social media and analytics APIs to enable seamless automation and performance tracking. The Meta Graph API is used for Facebook and Instagram post automation, insights, and engagement analytics. Additionally, the Analytics API is employed to monitor engagement and website performance metrics, and the YouTube Data API along with the YouTube Analytics API are used to analyze video performance, audience interaction, and content reach, ensuring a complete multi-platform analytics ecosystem.

2.1.6 Collaboration & Development Tools

Development & Version Control

- IDE / Code Editors**:** Visual Studio Code or PyCharm
- Version Control**:** GitHub for collaborative development and code management
- API Testing: Postman for designing and testing RESTful API endpoints
- Containerization (Optional): Docker for creating lightweight, portable containers ensuring consistent deployment environments

2.2 Product functions

1. User Registration and Authentication

- Allows users to **register** and **log in** securely using email/password or third-party authentication (Twitter, Facebook) through **OAuth 2.0**.
- Manages **user sessions** and stores tokens securely for connecting multiple social media accounts.
- Enables users to **link** or **unlink** their social media platforms within their profile settings.

2. Platform Integration

- Connects with multiple social media platforms (e.g., **Facebook, Instagram, Twitter/X**) through their **official APIs**.
- Fetches real-time **posts, comments, mentions, hashtags, and engagement metrics** from these platforms.
- Ensures data synchronization between user accounts and the system dashboard.

3. Data Collection and Preprocessing

- Periodically collects social media data (text, images, and engagement statistics).
- Performs data cleaning and preprocessing such as removing stopwords, emojis, and links before analysis.
- Stores the normalized data into the backend databases (MongoDB/PostgreSQL) for efficient retrieval.

4. Sentiment Analysis

- Applies NLP models (VADER or fine-tuned BERT-based classifiers) to analyze user posts and comments.
- Determines sentiment polarity (Positive, Negative, or Neutral) and assigns a confidence score.
- Displays graphical representation of sentiment trends over time through the analytics dashboard.

5. Trend Detection and Recommendation

- Continuously monitors hashtags, topics, and engagement keywords to identify emerging trends.
- Uses clustering or embedding-based techniques (e.g., TF-IDF, BERTopic, or FAISS) to group related topics.
- Suggests trending hashtags or content themes to improve reach and audience engagement.

6. AI-Based Caption and Hashtag Generation

- Utilizes **Generative AI models** (OpenAI API / Hugging Face Transformers) to create creative captions.
- Generates context-aware hashtags relevant to the post content and trending topics.
- Allows users to **edit, approve, or regenerate** captions before posting.

7. Post Scheduling and Automation

- Provides an interface to **schedule posts** for specific dates and times across multiple platforms.
- Uses background schedulers (Node-Cron / Celery) to automatically **publish posts** at the chosen time.
- Notifies users about posting status (success, failure, or pending) through alerts or dashboard updates.

8. Analytics and Reporting Dashboard

- Displays visual analytics using **Chart.js / Recharts / D3.js** libraries.
- Provides insights such as:
    - Engagement rate (likes, comments, shares)
    - Sentiment distribution (positive vs negative)
    - Top-performing posts and hashtags
    - Follower growth and reach trends
- Generates **custom reports** for selected time ranges and platforms.

9. Notifications and Alerts

- Sends notifications for:
    - Scheduled post confirmation
    - API token expiration or posting errors
    - Significant changes in engagement or sentiment trends
- Provides in-app notifications and optional email alerts.

3. Specific requirements

3.1 External interfaces

Inputs

1. APIs

I. OAuth Providers (Twitter, Facebook):
Used for secure authentication and user identity verification via OAuth 2.0.
Inputs include authorization codes and tokens.

II. Meta Graph API (Facebook & Instagram):
Retrieves user posts, comments, likes, and insights for analytics.
Also allows media uploads and scheduled post publishing.

III. Twitter/X Analytics APIs:
Fetch user posts, hashtags, mentions, and engagement metrics.
Enable automated post creation and data retrieval for dashboards.

IV. Web Scraping from Public Sources:
Collects additional trend and keyword data from public websites or social pages.
Inputs include target URLs, search keywords, and filtering parameters.

2. User Web Interface

Modern Web Dashboard that users interact with the system through a responsive web interface.
Inputs include:

- Authentication credentials (email/password or OAuth tokens)
- Post content (text, media, hashtags)
- Scheduling details (date, time, platforms)
- Filters for analytics (date range, platform, sentiment type)

All user inputs are validated before processing and securely transmitted via HTTPS.

Outputs (External)

1. APIs

I. Post Creation and Scheduling:
Generated or user-approved posts are published through integrated APIs (Meta GraphAPI, Twitter/X API).
Outputs include confirmation responses, post IDs, and publishing timestamps.

2. Web Interface

The Analytics Dashboard offers real-time insights into social media performance, showing key metrics like likes, comments, and shares. It also displays sentiment analysis results and trending hashtags or topics, helping users track audience engagement and refine their content strategy effectively.

The Visualization Components use Chart.js or D3.js to present data in an interactive and easy-to-understand format. Key visual elements include line charts for tracking performance over time, pie charts for displaying sentiment ratios, and tables summarizing top posts and overall engagement metrics.

Alerts and Notifications Real-time alerts for scheduled posts, API errors, or trend changes are shown directly in the dashboard.

3. Notification Services

The system provides automated notifications through push or email alerts to keep users Informed. These alerts notify users about completed post publishing, detected anomalies such as sudden drops in engagement, and token expiry or API failures, ensuring timely action and smooth platform operation.

3.2 Functional Requirements

FR1: User Registration and Authentication

1. The system shall allow new users to register using email and password or OAuth providers (Facebook, Twitter).
2. The system shall securely authenticate users before granting access to any dashboard or API function.

3. The system **s**hall validate login credentials and display appropriate error messages for invalid inputs.
4. The system shall encrypt and store access tokens securely using encryption standards
5. The system shall maintain session management and automatically log out users after a defined period of inactivity.

FR2: Platform Integration

1. The system shall enable users to link multiple social media platforms (Facebook, Instagram, Twitter).
2. The system shall retrieve user profile information, posts, comments, and analytics via official APIs.
3. The system shall validate API keys or tokens before data fetching or posting.
4. The system shall notify users if an API connection fails or a token expires.

FR3: Data Collection and Storage

1. The system shall collect data such as posts, comments, likes, shares, and hashtags from connected platforms.
2. The system shall preprocess collected text (e.g., removing URLs, emojis, and special characters).
3. The system shall store all retrieved and processed data in secure databases (MongoDB).
4. The system shall timestamp each collected record for chronological tracking.
5. The system shall automatically update collected data at defined time intervals (e.g., hourly, daily).

FR4: Sentiment Analysis

1. The system shall analyze text data from posts and comments using NLP-based models (VADER, BERT, etc.).
2. The system shall classify each text as Positive**,** Negative, or Neutral sentiment.
3. The system shall generate a confidence score for each sentiment prediction.
4. The system shall visualize aggregated sentiment data through charts and graphs on the dashboard.

FR5: AI-Based Caption and Hashtag Generation

1. The system shall generate captions and hashtags automatically using AI/NLP models (OpenAI, Hugging Face).
2. The system shall allow users to input a topic, keyword, or post description as a prompt for caption generation.
3. The system shall provide multiple AI-generated suggestions for each prompt.
4. The system shall allow users to modify or approve generated captions before scheduling or posting.
5. The system shall store the final approved caption and hashtags for reference in future analytics.

FR6: Trend Detection and Recommendation

1. The system shall identify trending hashtags, topics, and keywords from social data and web scraping.
2. The system shall group related hashtags and topics using clustering or frequency-based methods.
3. The system shall recommend trending topics to users based on their profile and engagement data.
4. The system shall update trend information in near real-time (e.g., every few hours).

FR7: Post Scheduling and Automation

1. The system shall enable users to schedule posts for multiple social platforms.
2. The system shall store scheduling details including post content, platform, and timestamp.
3. The system shall automatically publish the post at the scheduled date and time using API calls.
4. The system shall update post status as “Scheduled,” “Posted,” or “Failed” in the database.
5. The system shall retry posting in case of temporary API or network failures.
6. The system shall notify the user upon successful or failed posting.

FR8: Analytics and Insights Dashboard

1. The system shall display real-time statistics such as likes, comments, impressions, and shares.
2. The system shall compute and visualize engagement rate, sentiment distribution, and follower growth.
3. The system shall provide time-based filters (daily, weekly, monthly) for analytics visualization.
4. The system shall allow users to export analytics reports in CSV or PDF format.
5. The system shall update analytics data automatically without requiring a manual refresh.

FR9: Notification and Alert System

1. The system shall send alerts for scheduled post status, upcoming deadlines, or failed API connections.
2. The system shall notify users through in-app notifications and optional email alerts.
3. The system shall provide success messages for completed actions such as post publishing or trend updates.
4. The system shall maintain a notification log accessible through the user dashboard.

FR10: Admin and Maintenance Panel (Optional)

1. The system shall allow admin users to view and manage all registered user accounts.
2. The system shall enable the admin to approve or deactivate users if needed.
3. The system shall provide admin-level analytics such as total posts processed and API usage statistics.
4. The system shall log all system-level activities for monitoring and troubleshooting.

FR11: Data Backup and Recovery

1. The system shall automatically back up user data and analytics at regular intervals to cloud storage (AWS S3).
2. The system shall allow data restoration in case of accidental loss or server failure.
3. The system shall ensure versioning for critical user files and configuration data.

FR12: Security and Privacy Requirements

1. The system shall enforce HTTPS encryption for all data exchanges.
2. The system shall store all user credentials and tokens in encrypted format.
3. The system shall implement role-based access control (Admin/User).
4. The system shall comply with data privacy standards .
5. The system shall restrict unauthorized access to user accounts or private analytics data.

3.3 Performance Requirements :-

3.3.1 Static Numerical Requirements

Parameter	Requirement Description	Expected Value / Limit
Number of Supported Users	The system shall support multiple concurrent users accessing the platform.	Minimum **500 simultaneous users**; scalable up to **5,000+ users** with load balancing.
Number of Terminals / Devices	The system shall support concurrent access from web browsers, desktops, and mobile devices.	At least **1,000 active sessions** distributed across device types.
Database Capacity	The system shall handle data storage for posts, comments, analytics logs, and AI outputs.	Minimum **100 GB** of database storage; scalable up to **1 TB** using cloud databases (MongoDB Atlas, PostgreSQL).
API Integrations	The system shall maintain connections with social media APIs and AI services concurrently.	Minimum **4 active API integrations** (Meta, Twitter/X, OpenAI).
Backup Frequency	The system shall perform automated backups to cloud storage.	Minimum **once per 24 hours**; configurable based on usage.
Notification Delivery	The system shall handle alert and email notifications to users.	At least **500 notifications/hour** under normal operation.

3.3.2 Dynamic Numerical Requirements

Performance Metric	Requirement Description	Expected / Target Value
System Response Time	Time taken to respond to user actions (login, dashboard load, post scheduling).	Average ≤ **2 seconds**; Maximum ≤ **5 seconds** under peak load.
API Response Time	Average time to fetch or post data to external APIs (Instagram, Twitter, etc.)	≤ **3 seconds** (normal); ≤ **6 seconds** (peak conditions).
Data Processing Rate	Time required for preprocessing, sentiment analysis, and AI caption generation.	≤ **1.5 seconds per post** (average), ≤ **5 seconds** for longer content.
Post Scheduling Execution	Time taken by the scheduler to process and push queued posts.	Post executed within **±2 seconds** of scheduled time.
Analytics Refresh Rate	Frequency of data updates in dashboard analytics.	Real-time to every **5 minutes**, configurable by user.
Concurrent Transactions	Number of concurrent API calls or user actions supported by the backend.	Minimum **1,000 concurrent requests/transactions**.
Throughput	Total number of posts analyzed, generated, and published per hour.	Minimum **3,000 operations/hour** under normal conditions.
Peak Load Handling	System performance during high-traffic or bulk posting events.	Maintain ≥ **90%** normal throughput under **1.5× peak load**.
System Availability (Uptime)	Availability of the system for end users.	≥ **99.0% uptime** (monthly average).

3.3.3 Performance for Human Interaction

Interaction Type	Requirement Description	Expected Performance
Login and Authentication	Users should be authenticated quickly via OAuth or credentials.	Response ≤ **2 seconds**
Dashboard Loading	Initial loading of dashboard and analytics data.	≤ **3 seconds**
Caption Generation (AI)	AI model response time for text generation.	≤ **4 seconds** per prompt
Analytics Graph Interaction	Chart refresh when filters (date/platform) are applied.	≤ **2 seconds**

Notification Visibility	Time for alert or confirmation message to appear after an event.	≤ **1 second**

3.3.4 Performance Under Failure or Peak Conditions

1. The system shall maintain core functionalities (data collection, posting, analytics) even when one or more external APIs experience temporary downtime.
2. During network instability or heavy traffic, queued posts shall be retried automatically without data loss.
3. System shall recover to full operational performance within 30 seconds after reconnection or restart.
4. Logs of all delayed or failed transactions shall be maintained for auditing and debugging.

3.3.5 Scalability and Optimization

1. The system shall scale horizontally to handle increasing users and API load via cloud infrastructure.
2. The system shall use caching for frequent data queries to reduce latency.
3. The system shall balance workloads between servers using load balancers .
4. The system shall automatically scale resources based on real-time traffic (auto-scaling groups in cloud deployment).

3.5 Logical Database Requirements

The database is designed for efficient storage and frequent access of critical system data. User profiles and authentication details are accessed during login, while social media tokens and scheduled posts are regularly updated and retrieved. Sentiment and analytics data are queried during dashboard usage to provide real-time insights.

Data is organized with clear entities and relationships: users can have multiple social media accounts, each account can have many scheduled posts, and sentiment data is linked to individual posts. Integrity constraints like unique user emails, foreign key relationships, and

token validity checks ensure data consistency**.** Data retention policies maintain information during active or deactivated periods, with posts archived after publishing for future reference.

3.6 Logical Database Requirements

3.6.1 Reliability

Defines the software's ability to operate without failure over a specified period under expected conditions.
It must include error handling, fault tolerance, and recovery mechanisms to ensure continuous operation.
Example: Automatic rollback on failed transactions and maintaining consistent data states across sessions.

3.6.2 Availability

Specifies the expected uptime and accessibility of the system.
Includes features such as checkpointing, backup, recovery, and restart mechanisms to minimize downtime.
Example**:** 99.5% uptime with timely recovery from crashes or interruptions.

3.6.3 Security

Covers protection against unauthorized access, data breaches, and malicious activities.
Includes authentication, authorization, data encryption, and audit trails.
Example**:** Role-based access control, encrypted storage of sensitive user data, and secure API token management.

3.6.4 Maintainability

Describes how easily the software can be updated, fixed, or enhanced.
Includes modular design, clear documentation, and automated testing to simplify maintenance.
Example**:** Codebase structured for easy debugging and extension by development teams.

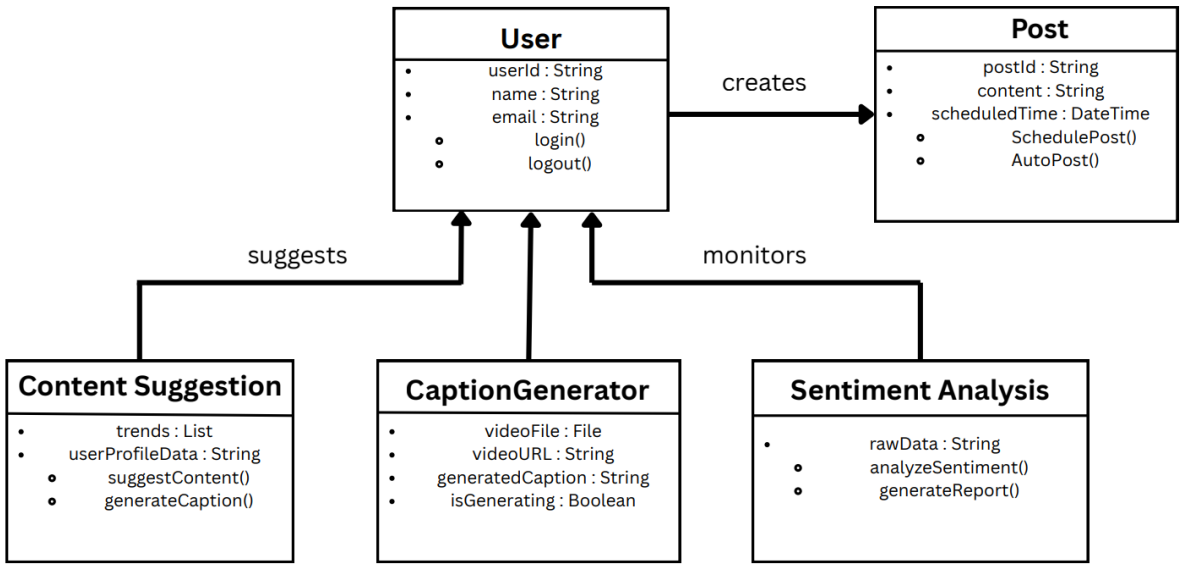
3.6.5 Portability

Defines the ease of transferring the software across different hardware or operating environments.
Relies on platform-independent code, standardized interfaces, and containerization.
Example**:** Deployable on cloud services with minimal configuration changes, supporting multiple operating systems and browsers.

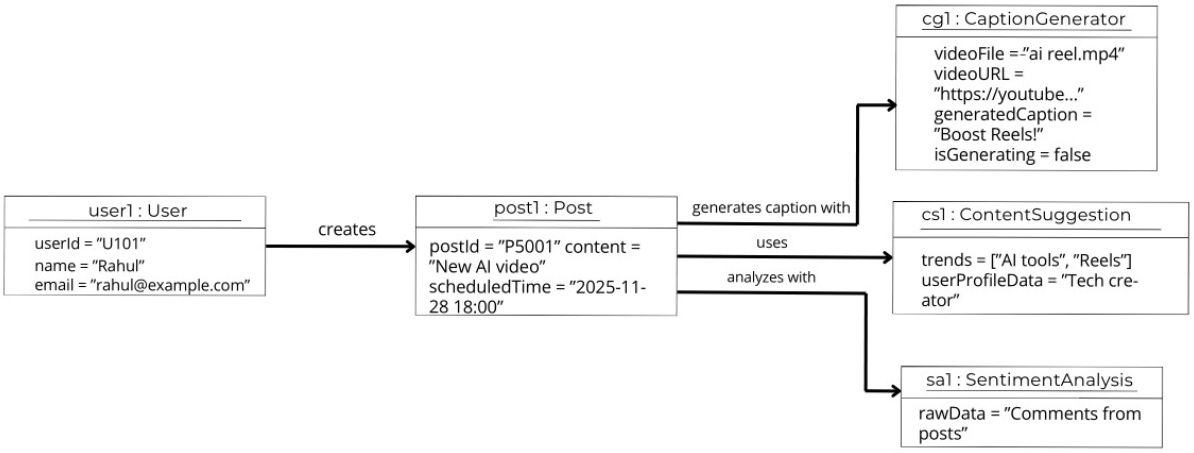
4. Software Design Document

4.1. Structural Design

a. Class Diagram



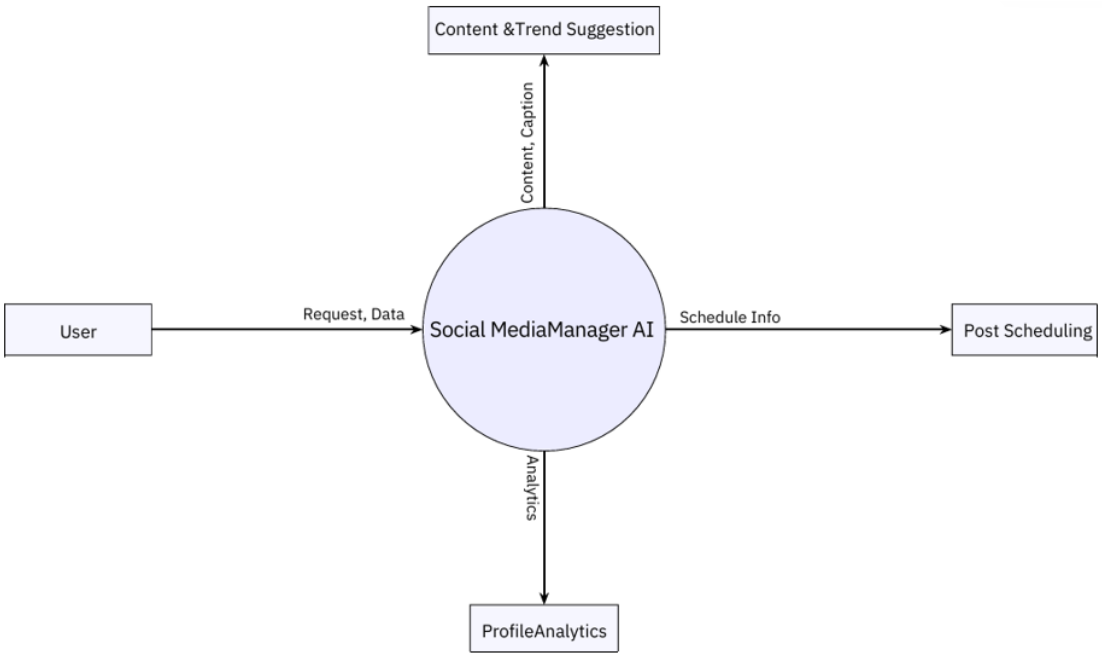
b. Object Diagram



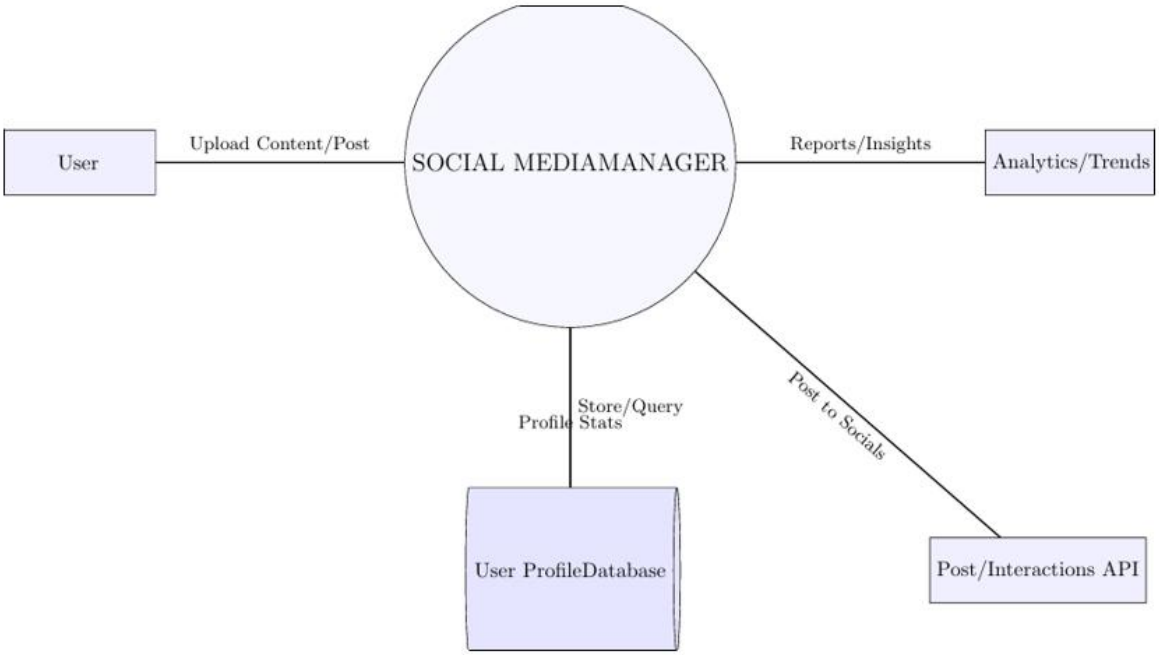
.

4.2. Data Design

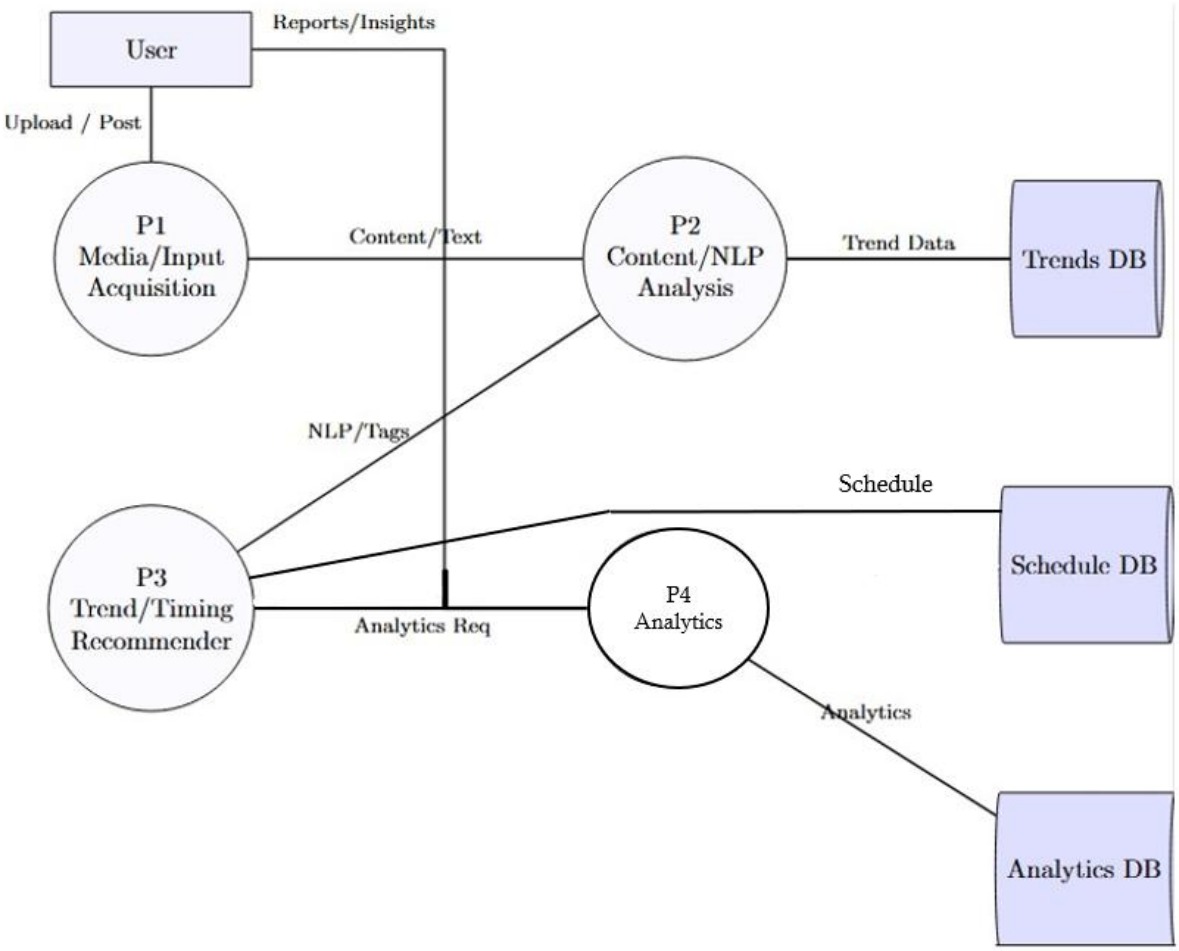
a. Data-flow Diagram(Level 0)



b. Data-flow Diagram(Level 1)

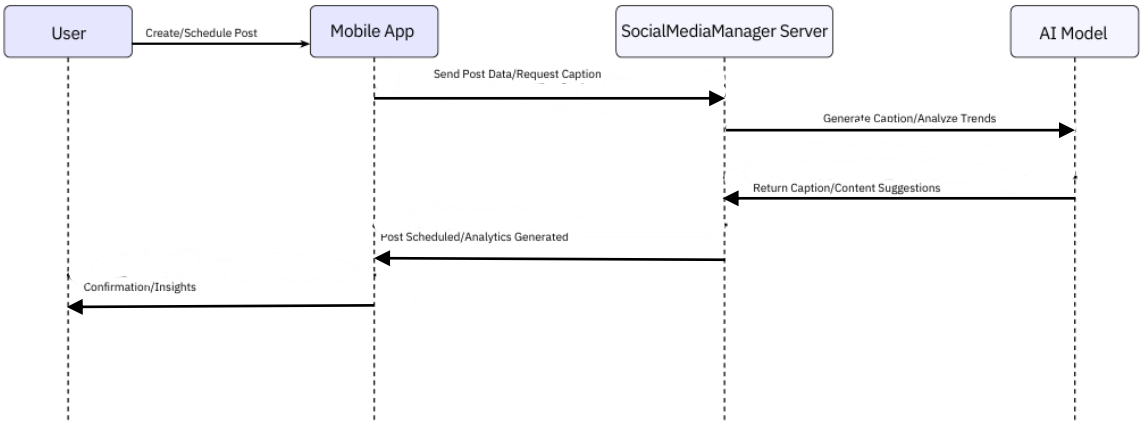


c. Data-flow Diagram(Level 2)

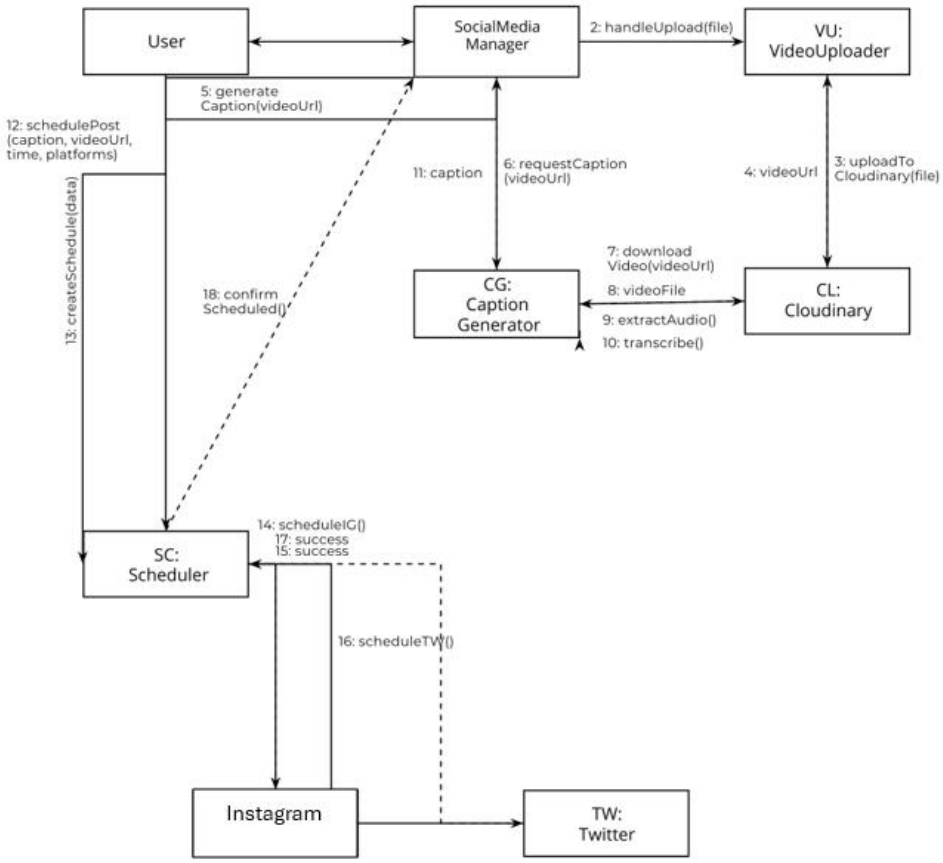


4.3. Behavioural Design

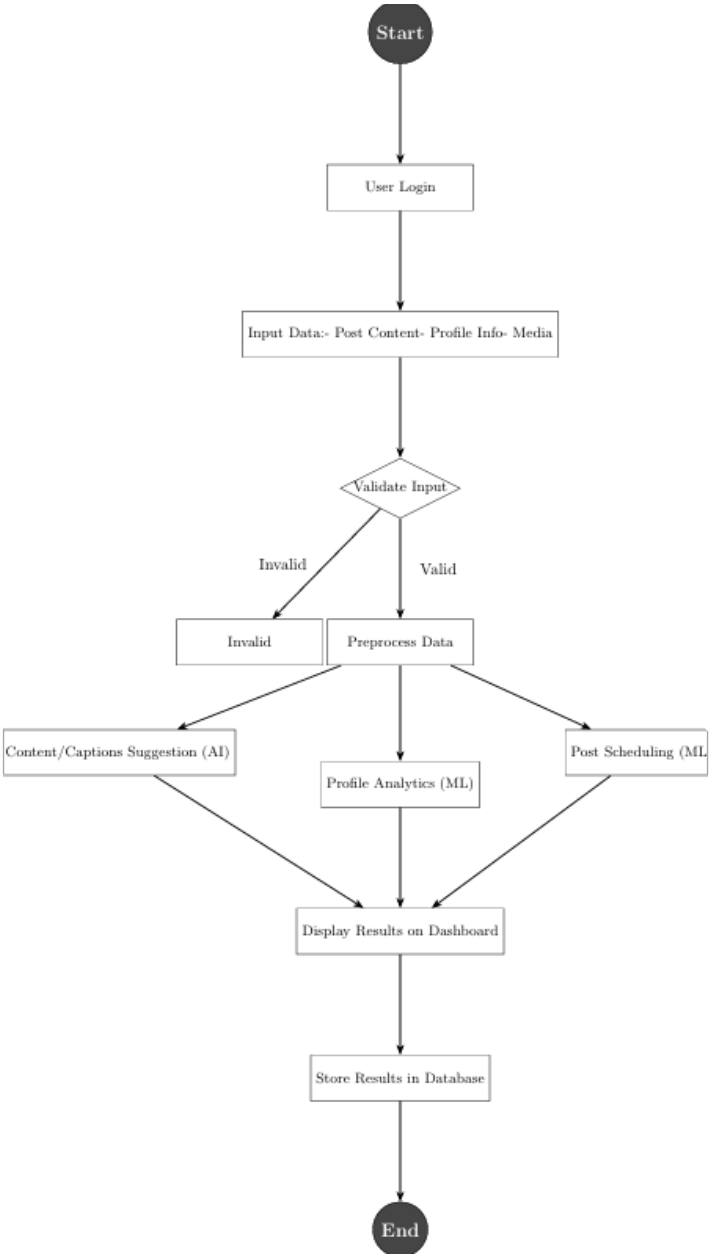
a. Sequence Diagram



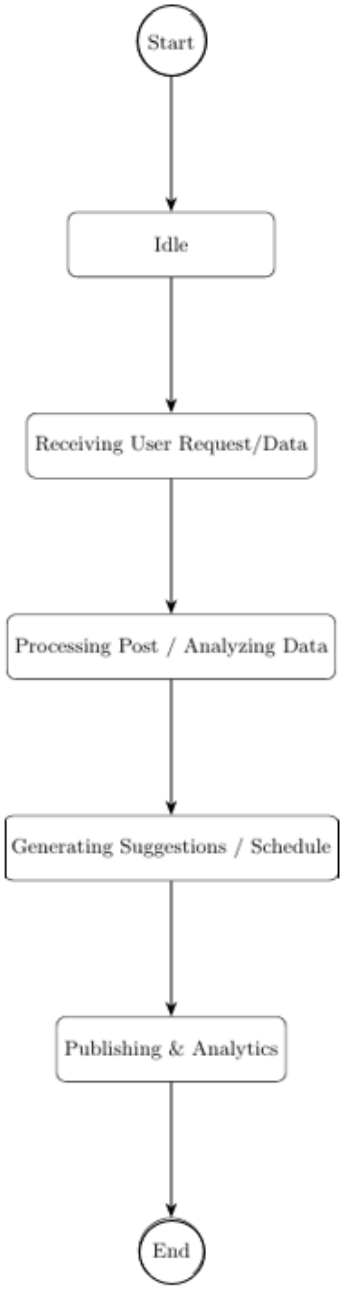
b. Collaboration diagram



c. Activity Diagram

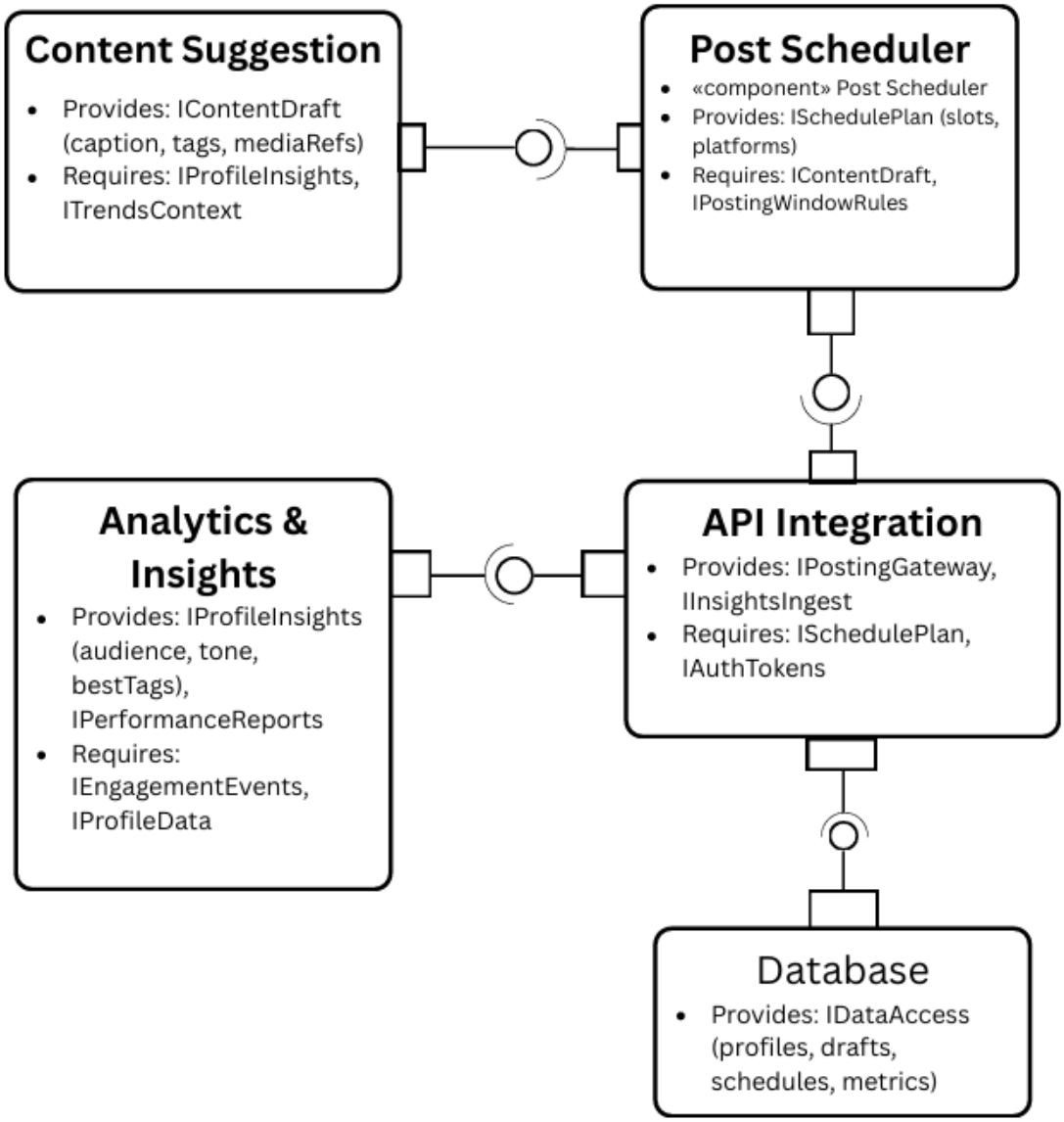


d. State-chart diagram



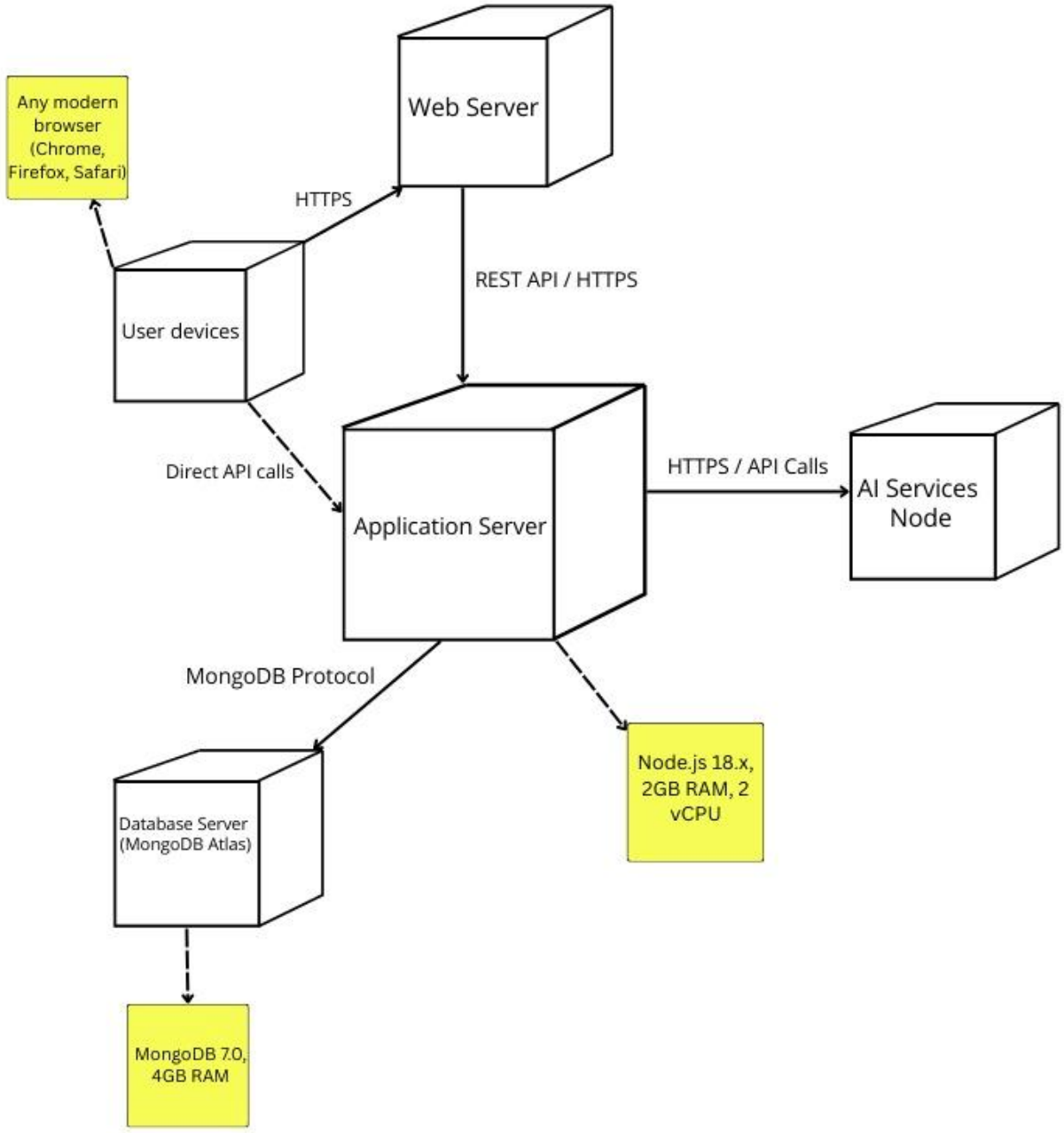
4.3. Implementation Design

a. Component Diagram



4.4. Environment Design

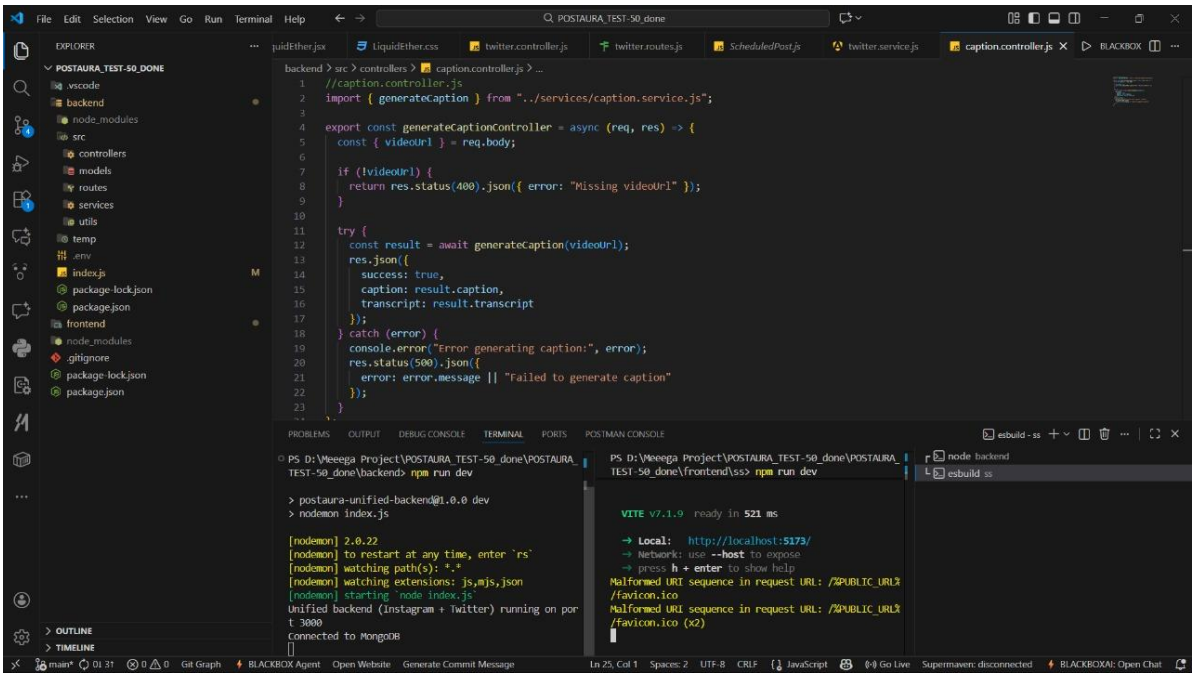
a. Deployment Diagram



References

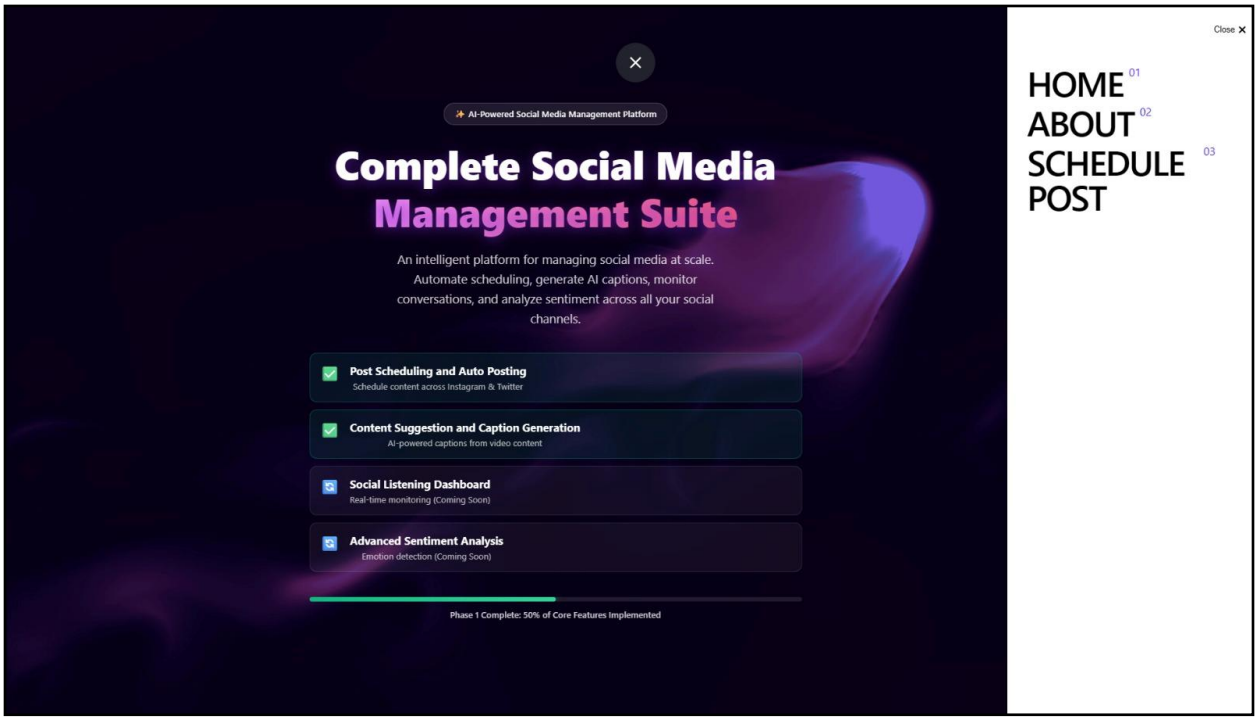
1. C. J. Hutto and E. E. Gilbert, "**VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text**" in Proceedings of the International Conference on Web and Social Media (ICWSM), 2014.
2. **Hootsuite**, "Social Media Listening Resources," 2024. [Online]. (Link)
3. L. Crummy, "**BeautifulSoup: Web Scraping with Python**," 2024. [Online]. (Link)
4. Google Developers, "**Google Analytics API Documentation**," 2024. [Online]. (Link)
5. Meta for Developers, "**Meta Graph API Documentation (Facebook & Instagram)**" 2024. [Online]. (Link)

Code

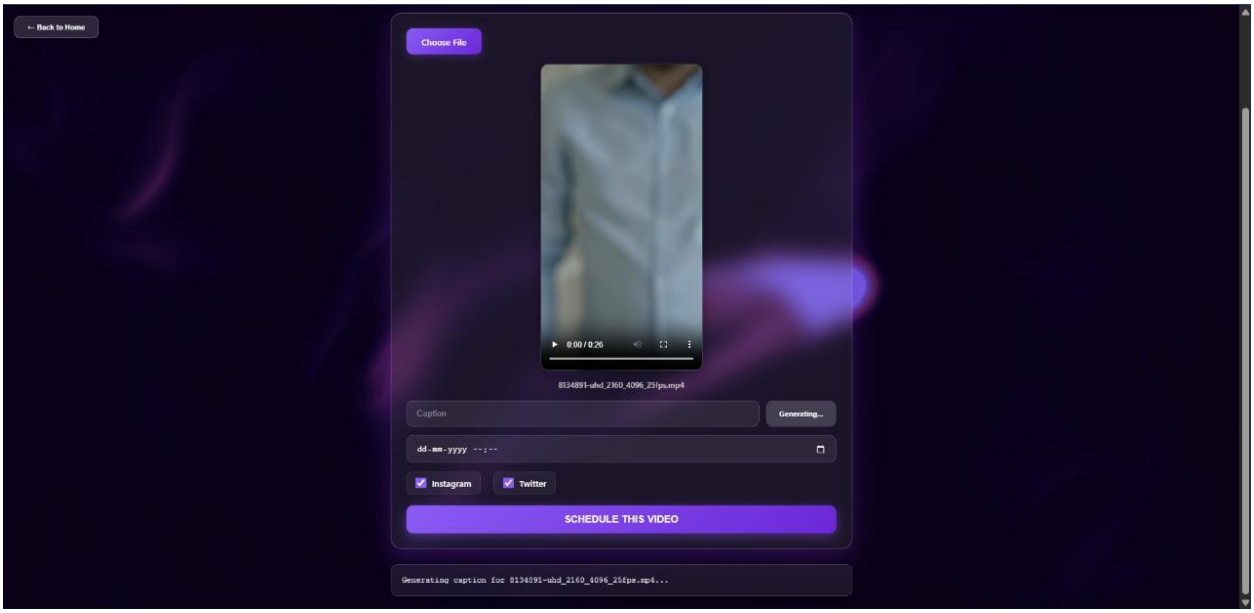

File Edit Selection View Go Run Terminal Help
POSTAURA_TEST-50_done
EXPLORER
POSTAURA_TEST-50_DONE
backend
frontend
caption.controller.js
backend > src > controllers > caption.controller.js > ...
//caption.controller.js
import { generateCaption } from "../services/caption.service.js";
export const generateCaptionController = async (req, res) => {
const { videoUrl } = req.body;
if (!videoUrl) {
return res.status(400).json({ error: "Missing videoUrl" });
}
try {
const result = await generateCaption(videoUrl);
res.json({
success: true,
caption: result.caption,
transcript: result.transcript
});
} catch (error) {
console.error("Error generating caption:", error);
res.status(500).json({
error: error.message || "Failed to generate caption"
});
}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS D:\Meeega Project\POSTAURA_TEST-50_done\POSTAURA_TEST-50_done\backend> npm run dev
> postaura-unified-backend@1.0.0 dev
> nodemon index.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
Unified backend (Instagram + Twitter) running on port 3000
Connected to MongoDB
PS D:\Meeega Project\POSTAURA_TEST-50_done\POSTAURA_TEST-50_done\frontend\ss> npm run dev
VITE v7.1.9 ready in 521 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
Malformed URI sequence in request URL: /%PUBLIC_URL%/favicon.ico
Malformed URI sequence in request URL: /%PUBLIC_URL%/favicon.ico (x2)
node backend
esbuild ss

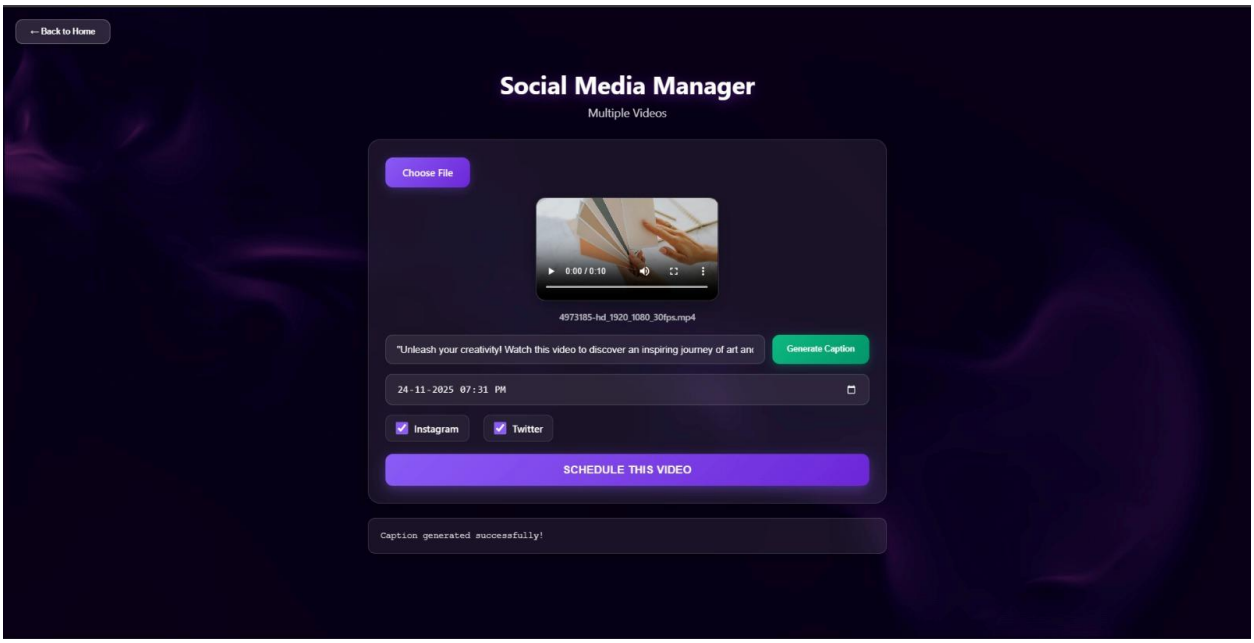
Outputs :-

a) Home Page :-



b) Caption Generation :-





c) Post Scheduling:-

